



Scikit-Learn (sklearn)

Machine Learning in Python Made Easy



Dated: 14 June 2025

Github: [bahadurCorvit](#)

Instructor : Fawad Bahadur

Agenda



- | | | | |
|----|--|----|-------------------------------------|
| 01 | Introduction | 05 | Evaluate models with proper metrics |
| 02 | Learn end-to-end ML pipeline | 06 | Use pipelines and cross-validation |
| 03 | Implement classification, regression, clustering | 07 | Tune models with GridSearchCV |
| 04 | Apply preprocessing techniques | 08 | Complete a mini ML project |

What is Scikit-Learn?

Scikit-Learn (sklearn) is an open-source Python library for:

-  Machine Learning
-  Data Mining
-  Data Analysis



Built on:

- NumPy
- SciPy
- Matplotlib

Key Features:

- Easy-to-use API
- Wide range of ML algorithms
- Integration with other tools (Pandas, NumPy)



Why Use Scikit-Learn?

Advantages of Scikit-Learn:

- Simple and consistent API for all ML tasks
- Interoperable with other Python libraries (Pandas, NumPy)
- Efficient implementation of many ML algorithms
- Extensive documentation and community support
- Built-in tools for model evaluation and selection



Ideal for:

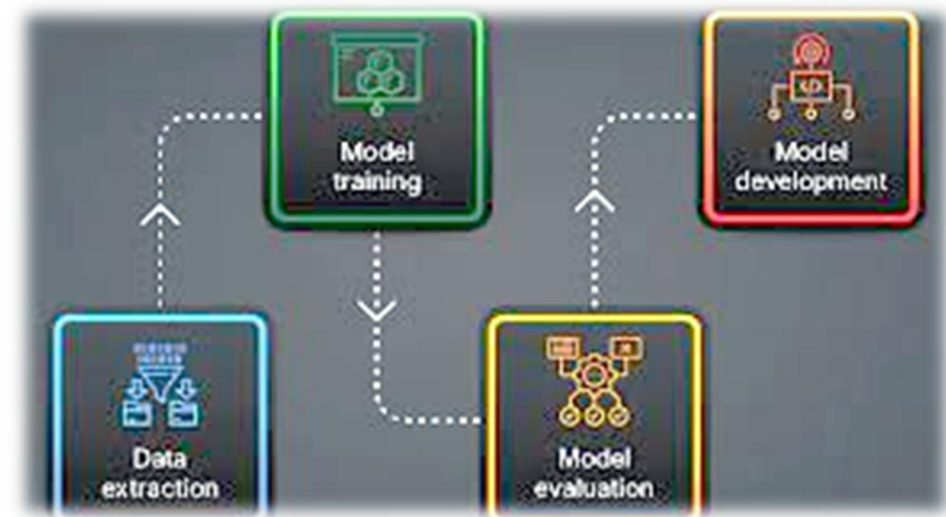
Beginners & advanced users
Prototyping ML models quickly



Scikit-Learn Workflow

Typical ML Pipeline in Scikit-Learn:

1. Import Libraries
2. Load Dataset
3. Preprocess Data
4. Split Data (Train/Test)
5. Choose a Model
6. Train the Model
7. Make Predictions
8. Evaluate Performance
9. Optional: Hyperparameter Tuning & Cross-Validation



Installation

Requirements:

Python ≥ 3.8

NumPy, SciPy (automatically installed)

Using pip:

```
pip install scikit-learn
```

Using conda:

```
conda install scikit-learn
```

Check installation:

```
import sklearn  
print(sklearn.__version__)
```



Importing Scikit-Learn

Typical Imports in scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```



Easy to integrate with:

- NumPy for numerical operations
- Pandas for data manipulation
- Matplotlib/Seaborn for visualization



Built-in Datasets

Scikit-Learn Provides Sample Datasets

```
from sklearn.datasets import load_iris
data = load_iris()
print(data.feature_names)
```



Popular Built-in Datasets:

- `load_iris()` – flower classification
- `load_digits()` – handwritten digits
- `load_diabetes()` – regression task
- `load_breast_cancer()` – binary classification



Data Preprocessing in Scikit-Learn

Why Preprocess?

To ensure data is clean, consistent, and suitable for training.

Missing values

PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	male	22	1	0	A/5 21171	7.25		S
2	1	1	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	female	26	0	0	STON/O2. 3101282	7.925		S
4	1	1	female	35	1	0	113803	53.1	C123	S
5	0	3	male	35	0	0	373450	8.05		S
6	0	3	male		0	0	330877	8.4583		Q

Common Preprocessing Tasks:

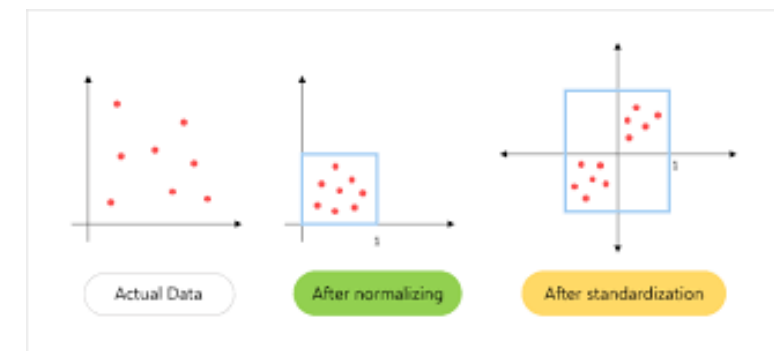
- Handling missing values
- Encoding categorical features
- Feature scaling/normalization

Categorical Feature Encoding

Colors	Yellow	Blue	Green
Yellow	1	0	0
Blue	0	1	0
Green	0	0	1

Key Modules:

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer



Train-Test Split

Why Split Data?

To evaluate the model's performance on unseen data.

Function Used:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

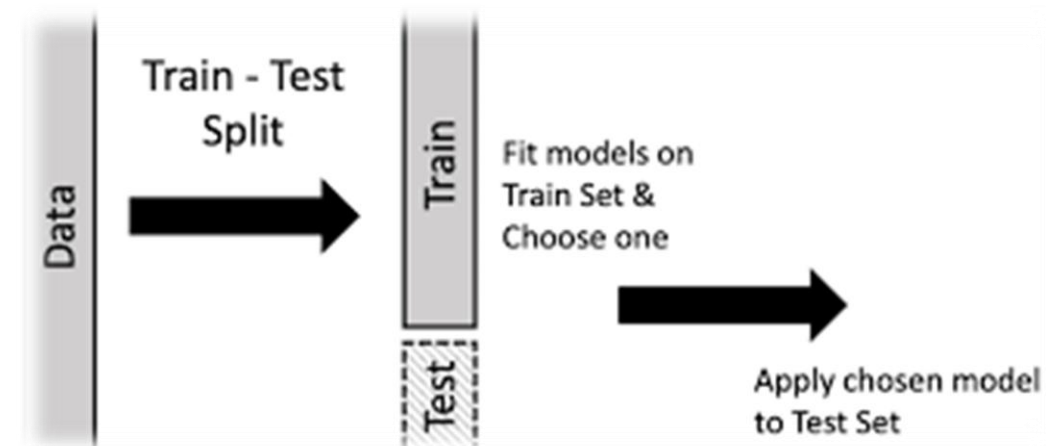
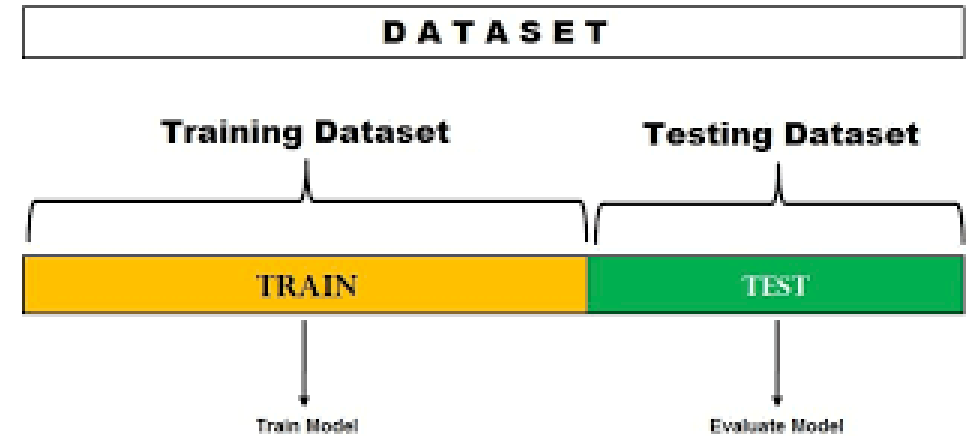
Parameters:

test_size: Proportion of test data (e.g., 0.2 = 20%)

random_state: Ensures reproducibility

Goal:

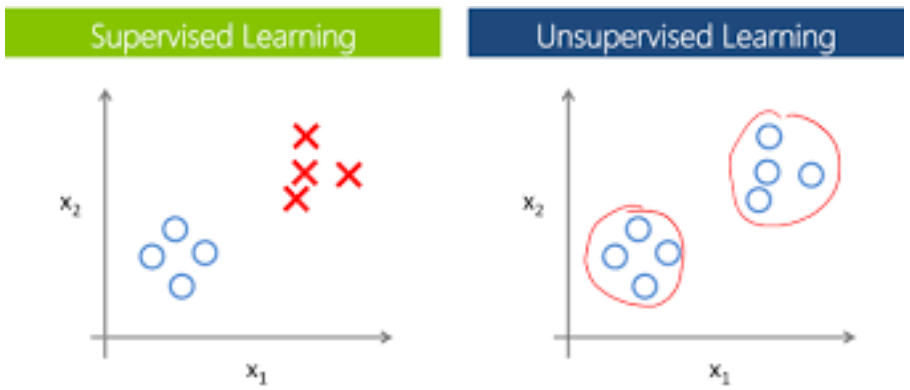
Prevent overfitting by using separate test data



Supervised vs Unsupervised Learning

Two Main Types in Scikit-Learn

-  **Supervised Learning:**
Learn from labeled data
-  **Unsupervised Learning:**
Discover hidden patterns



Type	Description	Examples
Supervised	Input and labeled output	Classification, Regression
Unsupervised	Only input data, no labels	Clustering, Dimensionality Reduction

Supervised Learning – Classification

What is Classification?

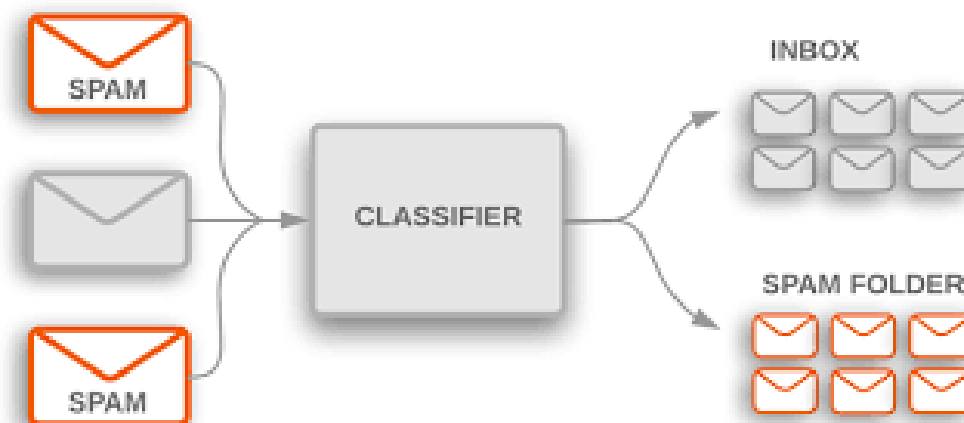
Predicting a **discrete** label or category.

Popular Classification Models in Scikit-Learn:

- LogisticRegression
- KNeighborsClassifier
- DecisionTreeClassifier
- RandomForestClassifier
- SVC (Support Vector Classifier)
- NaiveBayes (GaussianNB)

Example Use Cases

- Email spam detection
- Disease diagnosis
- Image recognition



Supervised Learning – Regression

What is Regression?

Predicting a continuous numerical value.

Popular Regression Models in Scikit-Learn:

- LinearRegression
- Ridge, Lasso
- SVR (Support Vector Regression)
- DecisionTreeRegressor
- RandomForestRegressor

Example Use Cases

- House price prediction
- Stock price forecasting
- Sales estimation



Unsupervised Learning – Clustering

What is Clustering?

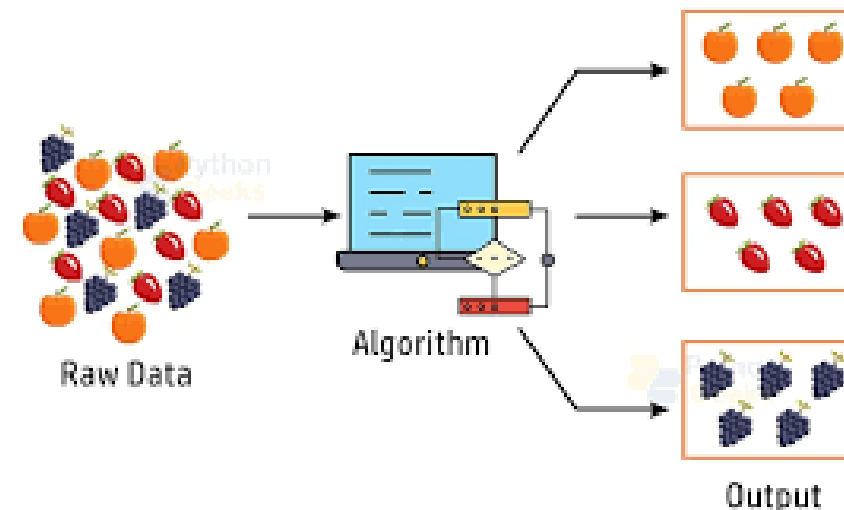
Grouping similar data points without labels.

Popular Clustering Algorithms in Scikit-Learn:

- KMeans
- DBSCAN
- AgglomerativeClustering

Example Use Cases

- Customer segmentation
- Social network analysis
- Market basket analysis



Unsupervised Learning – Dimensionality Reduction

What is Dimensionality Reduction?

Reducing the number of input features while preserving important information.

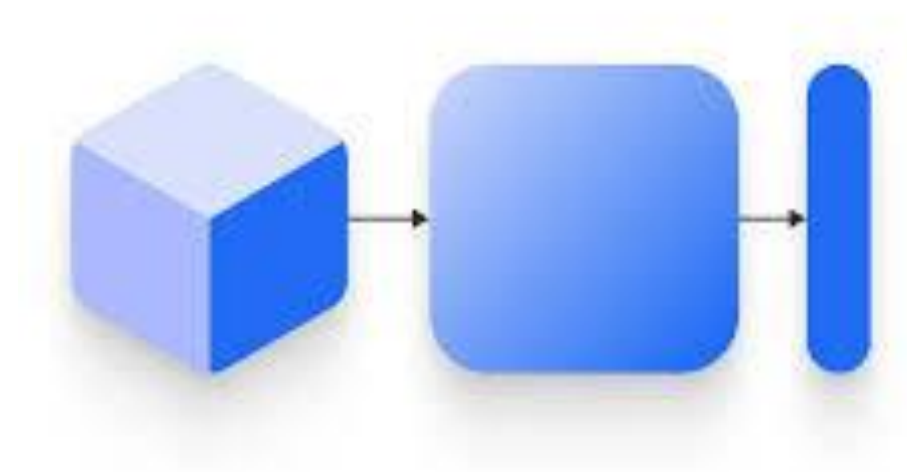
Popular Techniques in Scikit-Learn:

- PCA (Principal Component Analysis)
- TruncatedSVD
- t-SNE (for visualization)

Example Use Cases

- Visualizing high-dimensional data
- Speeding up ML models
- Removing multicollinearity

Fewer dimensions → Faster training + Less overfitting



Model Selection in Scikit-Learn

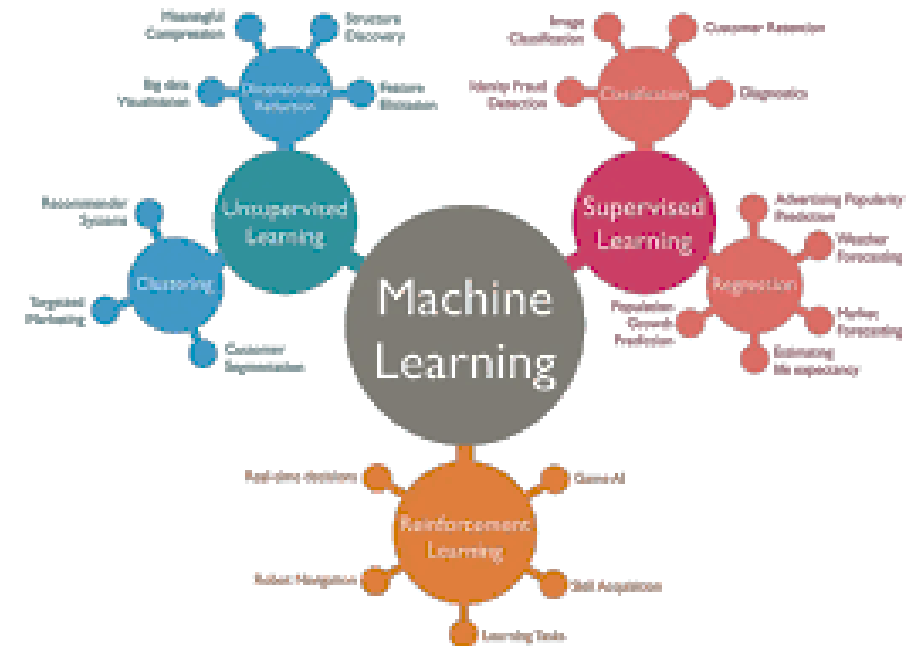
Choosing the Right Model

Depends on:

- Type of problem (classification, regression, clustering)
- Size and nature of dataset
- Performance goals (accuracy, speed, interpretability)

Scikit-Learn Helps With:

- Wide variety of models
 - Easy model switching
 - Same interface: `.fit()`, `.predict()`
- Try multiple models & compare performance!



Training a Model

Basic Steps to Train a Model

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

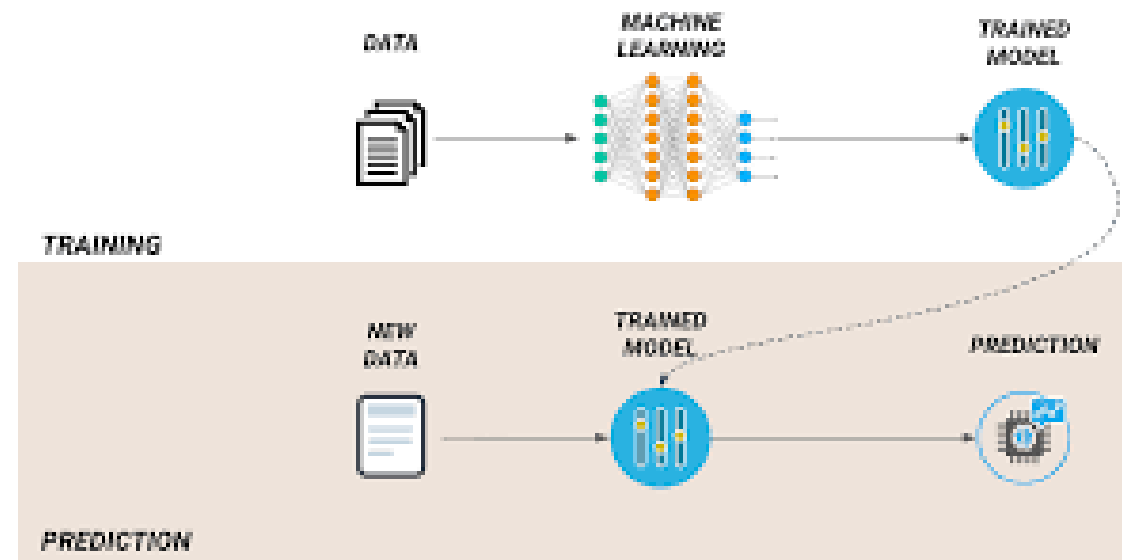
What Happens During .fit()?

- Learns the relationship between input X_{train} and output y_{train}
- Stores model parameters (like weights)

All models in sklearn follow:

```
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)
```



Making Predictions

Once Trained, Use the Model to Predict

```
y_pred = model.predict(X_test)
```

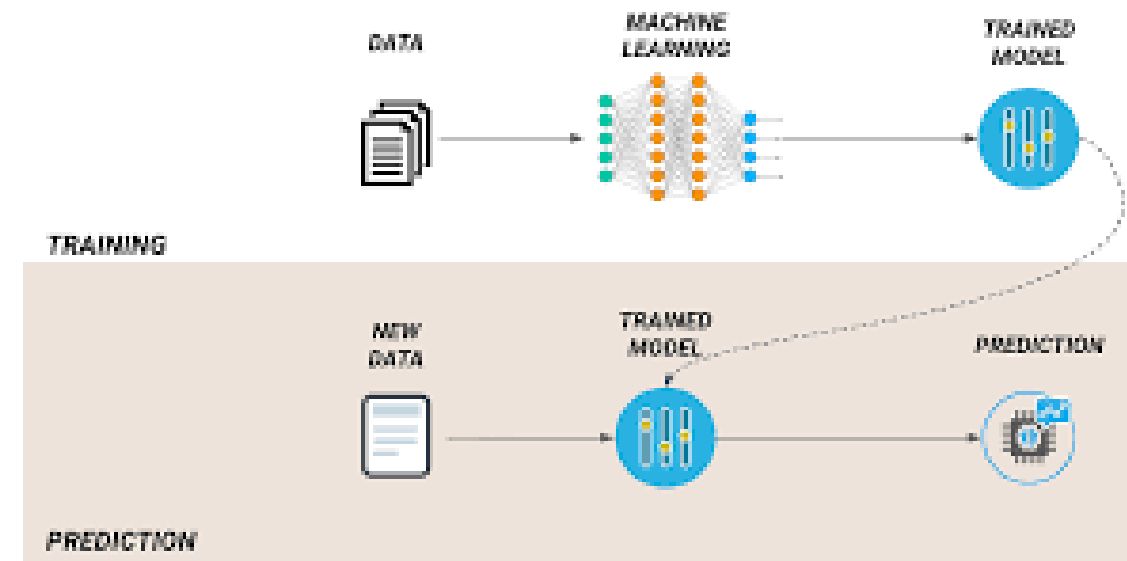
What Happens?

- The model uses learned parameters
- Applies them to unseen X_{test} data
- Returns predicted values (y_{pred})

Used for:

Evaluating performance

Making real-world decisions



☒ Works the same way for classification & regression

Model Evaluation – Classification Metrics

How Well Did the Model Perform?

Key Classification Metrics:

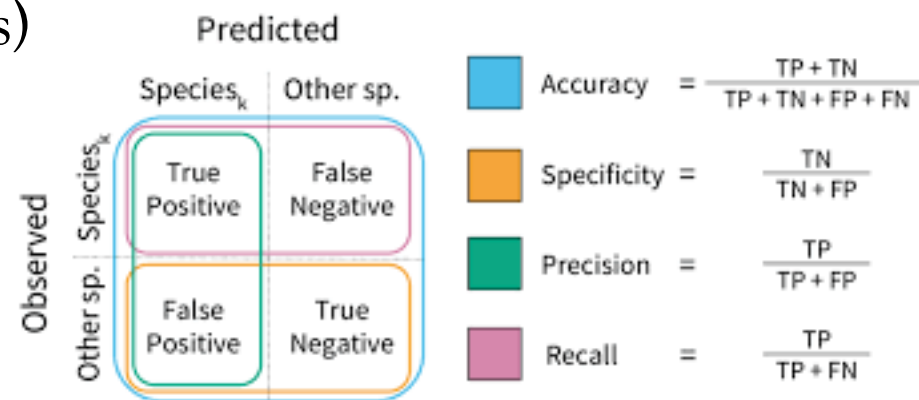
Accuracy – Correct predictions / Total predictions

Precision – True Positives / (True Positives + False Positives)

Recall – True Positives / (True Positives + False Negatives)

F1 Score – Harmonic mean of precision and recall

Confusion Matrix – Visual performance breakdown



Scikit-Learn Tools:

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix
```

Evaluate Regression Models Using These Metrics:

Mean Absolute Error (MAE)

Average of absolute differences between predicted and actual values

Mean Squared Error (MSE)

Average of squared differences

Root Mean Squared Error (RMSE)

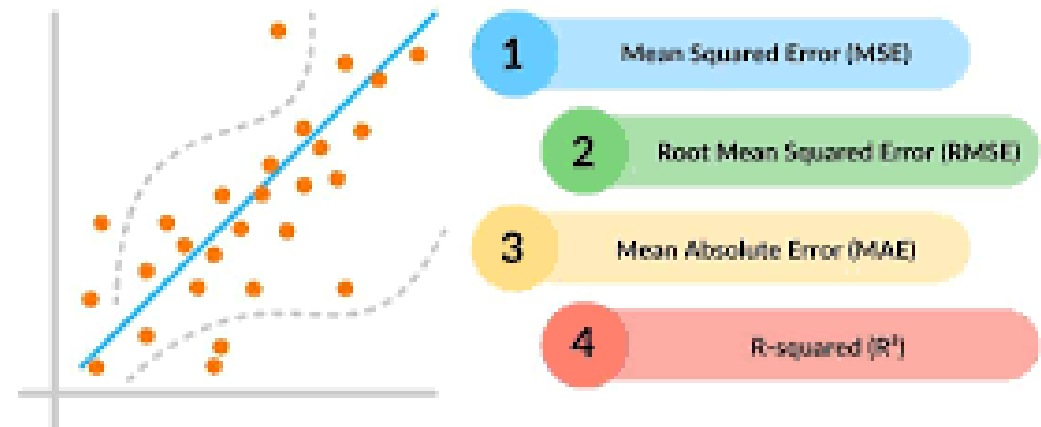
Square root of MSE

R^2 Score (Coefficient of Determination)

Proportion of variance explained by the model

Scikit-Learn Example:

```
from sklearn.metrics import mean_squared_error, r2_score
```



What is Cross-Validation?

A technique to evaluate model performance more reliably by splitting data into multiple train-test folds.

Scikit-Learn Function:

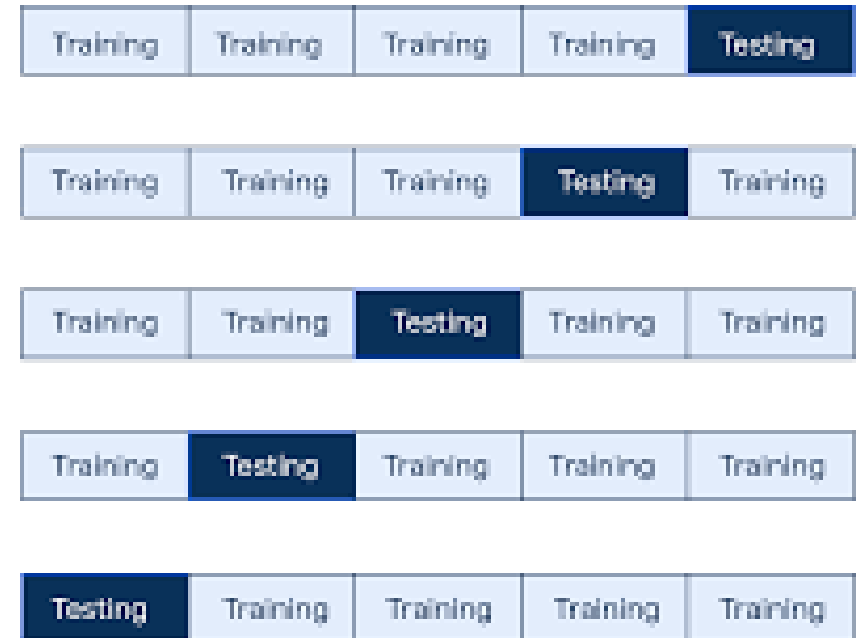
```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(model, X, y, cv=5)
```

Benefits:

- Reduces overfitting risk
- Gives a better estimate of generalization
- Works with any supervised model

cv=5 means 5-fold cross-validation



Hyperparameter Tuning

What Are Hyperparameters?

Settings passed before training (e.g., number of neighbors in KNN)

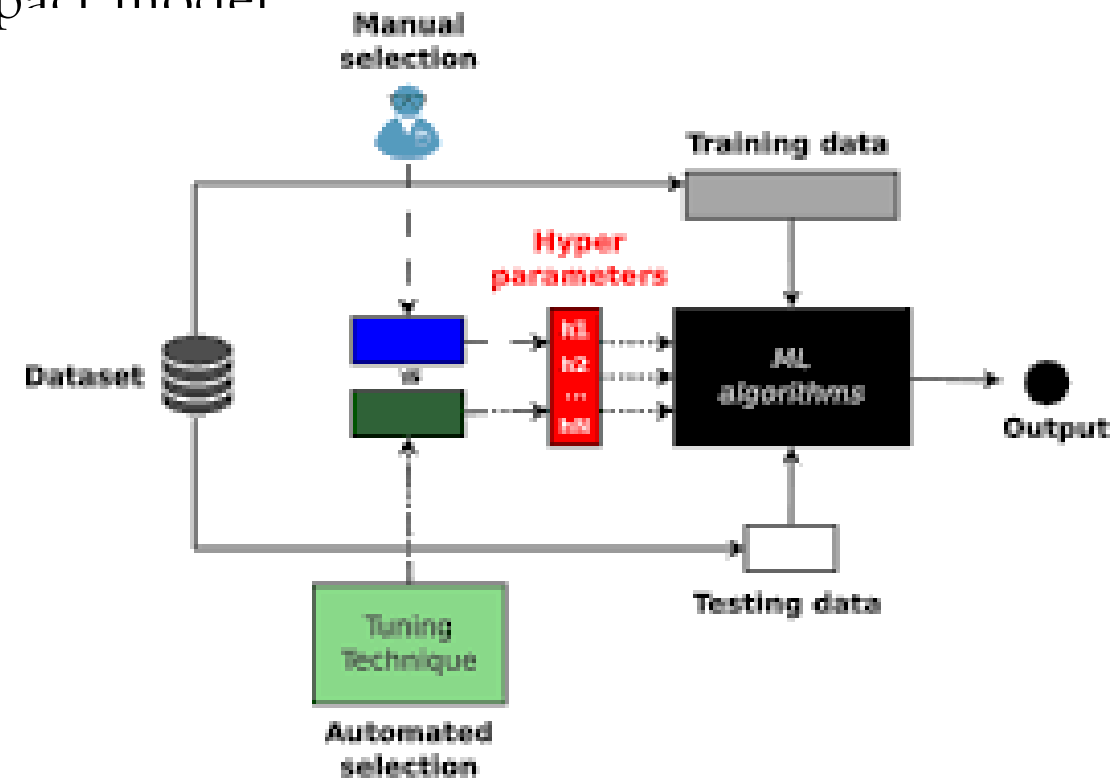
✂ Not learned during training, but significantly impact model performance.

 **Tuning Methods in Scikit-Learn:**
GridSearchCV – Exhaustive search
RandomizedSearchCV – Random combinations

```
from sklearn.model_selection import GridSearchCV
```

```
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
```

Find the best combination of parameters



Pipelines in Scikit-Learn

Why Use Pipelines?

Combine multiple steps (e.g., preprocessing + modeling) into a **single object**.

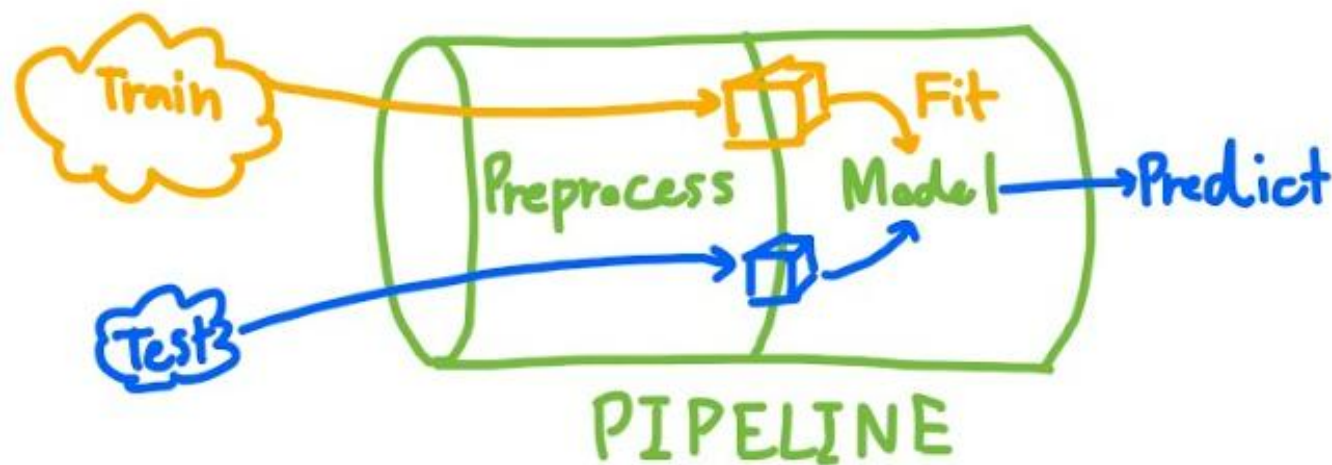
```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression())
])
pipe.fit(X_train, y_train)
```

-  **Benefits:**
- Clean code
- Reproducibility
- Reduces data leakage



Why Scale Features?

Some ML algorithms are sensitive to feature scales (e.g., KNN, SVM, Gradient Descent-based models)

Common Scalers in Scikit-Learn:

StandardScaler: Mean = 0, Std = 1

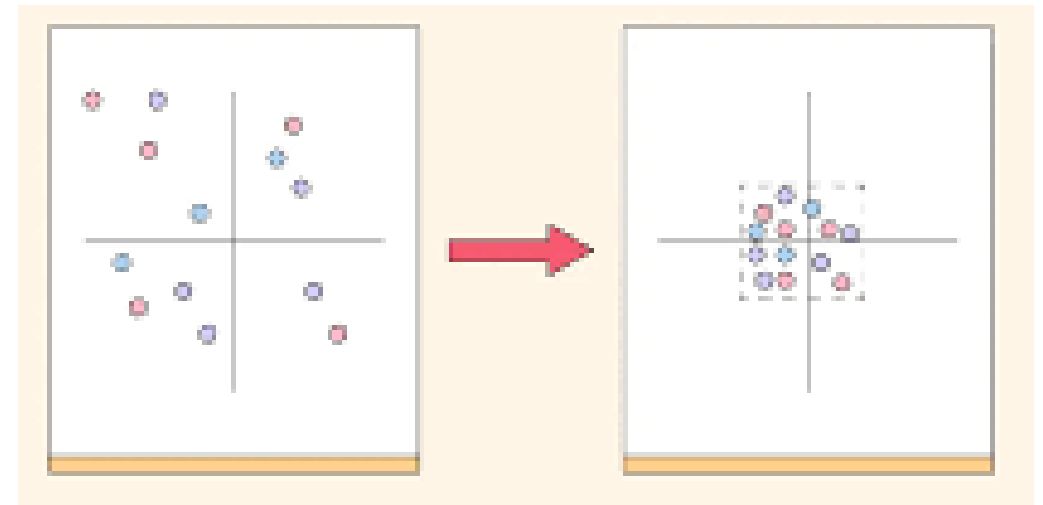
MinMaxScaler: Scales between 0 and 1

RobustScaler: Handles outliers well

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```



 Always fit scaler only on training data!

Handling Categorical Data

Why Handle Categorical Features?

ML models work with numbers, not strings.

Techniques in Scikit-Learn:

LabelEncoder: Converts labels to numbers

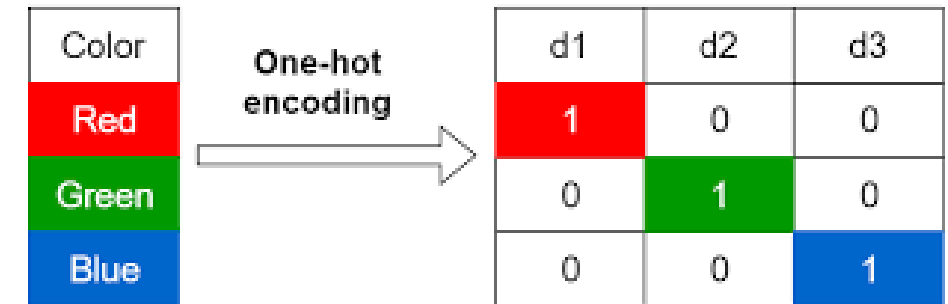
OneHotEncoder: Creates binary columns for each category



```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder = OneHotEncoder()
```

```
X_encoded = encoder.fit_transform(X_categorical)
```



 Use ColumnTransformer to apply encoders selectively in pipelines.

Handling Missing Data

Why Handle Missing Values?

Missing values can break model training or reduce accuracy.

Scikit-Learn Solution: SimpleImputer

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy='mean') # or 'median',  
'most_frequent'
```

```
X_imputed = imputer.fit_transform(X)
```

Strategies:

Mean/Median for numerical data

Most frequent for categorical data

Constant value (e.g., 0 or “Unknown”)

 Use in Pipelines for automation!



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

Diagram showing the transformation of missing values (NaN) in the 'price' column to their mean value (1.2 for apple, 3.1 for mango) using the 'mean' strategy.

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	1.2
2	2021-01-03	apple	1.2
3	2021-01-04	apple	1.2
4	2021-01-01	mango	3.1
5	2021-01-02	mango	3.1
6	2021-01-03	mango	2.8
7	2021-01-04	mango	2.8

Saving and Loading Models

 Preserve Trained Models for Later Use

 Using joblib (preferred for sklearn objects):

```
import joblib
```

```
# Save model
```

```
joblib.dump(model, 'model.pkl')
```

```
# Load model
```

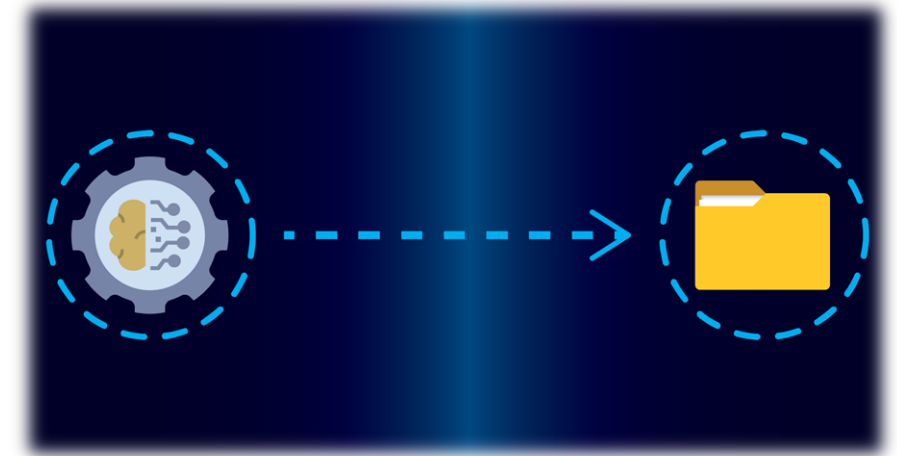
```
model = joblib.load('model.pkl')
```

 Useful for:

Reuse without retraining

Deployment in production systems

Sharing models with others



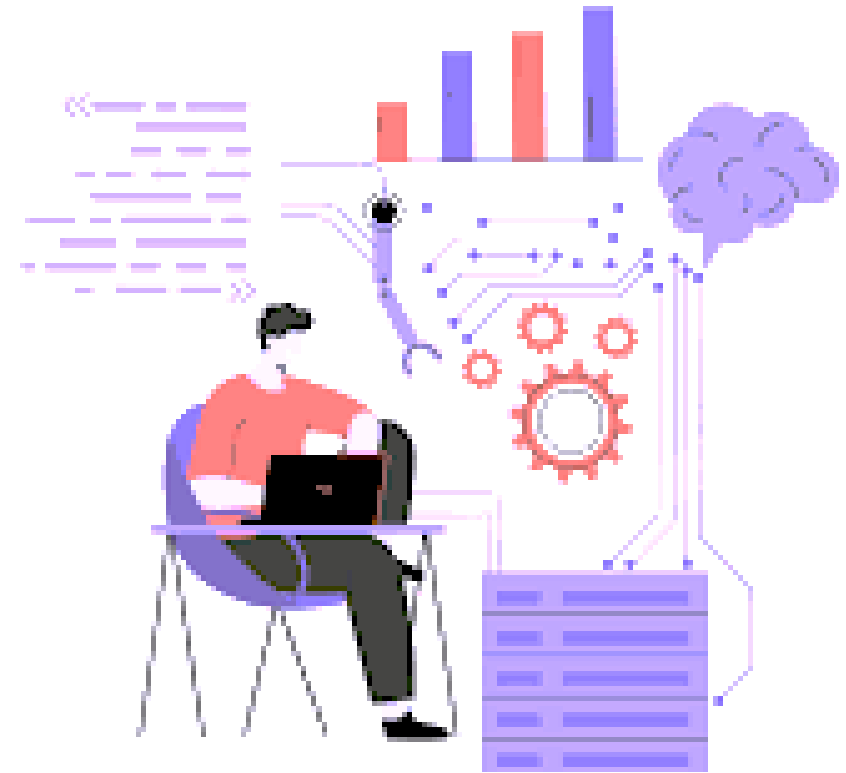
Real-World Project Example



 **Use Case:** Predicting House Prices

Steps:

1. Load dataset (e.g., Boston Housing)
2. Preprocess features (scale, handle missing)
3. Split into train/test sets
4. Train LinearRegression model
5. Evaluate using MSE & R^2
6. Tune hyperparameters (if needed)
7. Save final model



 **Combines:** Regression, Preprocessing, Pipeline, Evaluation

 Encourages end-to-end ML workflow using Scikit-Learn

Integration with Other Libraries

Scikit-Learn Works Seamlessly With:

-  NumPy – Numerical arrays
-  Pandas – DataFrames for structured data
-  Matplotlib / Seaborn – Visualization
-  XGBoost, LightGBM – Advanced models using sklearn API
-  TensorFlow / PyTorch – Deep learning + sklearn for preprocessing
-  Many tools are Scikit-Learn compatible via `.fit()` / `.transform()` API
-  Scikit-Learn = Core of the Python ML ecosystem



TensorFlow



Common Challenges & Best Practices



⚠ Common Pitfalls:

- 1) Not scaling data when needed
- 2) Data leakage (using test data in training)
- 3) Overfitting from too complex models
- 4) Ignoring class imbalance

✅ Best Practices:

- i. Use Pipelines to prevent leakage
- ii. Apply Cross-Validation for reliable metrics
- iii. Tune hyperparameters systematically
- iv. Monitor overfitting with validation set
- v. Document & version control your models



Summary and Final Thoughts



Key Takeaways:

- Scikit-Learn is a **powerful, easy-to-use** ML library in Python
- **Supports full ML pipeline:** preprocessing → modeling → evaluation → deployment
- Offers a wide range of algorithms for both **supervised** and **unsupervised learning**
- Encourages best practices via **pipelines, cross-validation, and grid search**
- Integrates well with tools like **Pandas, NumPy, Matplotlib, and joblib**



Next Steps:

- Explore real datasets
- Try out various models
- Build and deploy your own ML project!





**THANK
YOU**